PROCESSING COMMUNITY DAY 2026

@BURSA

```
for (int i = 0; i < 1000; i++) {
    float x = noise(i * 0.01) * width;
    float y = noise(i * 0.01 + 100) * height;
    stroke(0, 255, 0);
    line(x, y, x + random(-5, 5), y + random(-5, 5));
}
```

code
random
line
machine
art

010110
011011
011100
001000
010011
011101
011030

P5.js / Processing ile Yaratıcı Kodlama

Algoritmik Tasarım ve Generatif Sanat

YARATICI KODLAMA ATÖLYESİ

■ Atölye Yürütücüsü: Bedirhan Uğur

NİLÜFER BELEDİYESİ BURSA
İNOVASYON MERKEZİ

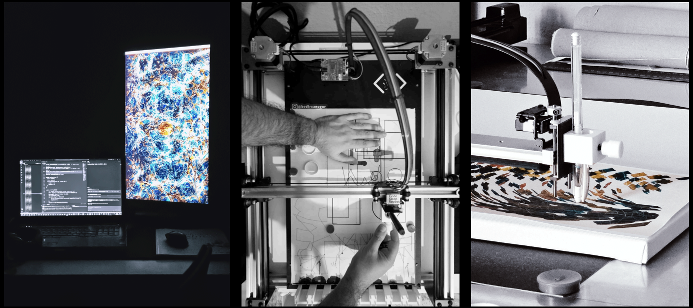
Atölye Yürütücüsü

Ben Bedirhan Uğur, Sanatçı ve Yaratıcı Teknoloji Geliştiricisiyim.

- Generatif Sistemler, Yaratıcı Kodlama ve Çizim Robotları
- Algoritmik Tasarım ve İnsan-Makine Etkileşimi
- Processing, p5.js, openFrameworks, Raspberry Pi

Daha Fazlası için Ziyaret Edin

bedirxanugur.com ↗



Hoş Geldiniz!

Kısaca Tanışalım mı?

İsim + /* Kodlama, Sanat, Tasarım ve Teknoloji */ ile Aranız Nasıl?

(Yeni / Meraklı / Üretiyor)

Bugünün Hedeflenen Programı

Saat	Bölüm
13:00–13:15	Karşılama ve Tanışma 👋
13:15–13:25	PCD Hakkında
13:25–13:45	Yaratıcı Kodlama Nedir?
13:45–14:20	p5.js Temelleri **▣ Rehber**
14:20–14:30	**Ara** ☕

Saat	Bölüm
14:30–15:20	Uygulamalı Çalışma **▤ Kitapçık**
15:20–15:30	**Ara** ☕
15:30–15:45	İnsan · Algoritma · Makine
15:45–16:20	Çizim Robotu Gösterimi 🦾
16:20–17:00	Soru-Cevap ve Tartışma 💬

▣ Rehber Türkçe Bilgi Rehberi **▤ Kitapçık** Uygulama Kitapçığı

Bugün Ne Yapacağız?

- **Processing Açık Kaynak Kodlama Topluluğunu Tanıyacağız**
  Topluluk Yapısı · Yerel ve Uluslararası Etkinlikler
- **p5.js ile Yaratıcı Kodlamanın Temellerini Kuracağız**
  Türkçe Bilgi Rehberi ile Birlikte · Editör · Koordinat · Renk · Şekil · Rastgelelik
- **Beraber Kısa Bir Görsel Algoritmik Sistem Üreteceğiz**
  Kitapçık ile Birlikte Uygulamalı Çalışma
- **Koddan Çizgiye: Algoritmaların Makineyle Buluşmasını İzleyeceğiz**
  Çizim Robotu ile Canlı Çizim · Algoritmalar ile Makine Kontrolü

Bugün Kullanacaklarımız

- **p5.js Web Editor** — Tarayıcıda (Birincil)
- **Processing** Masaüstünde
- Türkçe Hızlı Başvuru Rehberi
- Uygulama Kitapçığı

PCD Hakkında

Processing Community Day 2026
Processing Foundation

A Quarter Century of Processing | Ars Electronica 2026 ↗

Processing Community Day Nedir?

- Processing Foundation Destekli Uluslararası Etkinlik Ağı
- Sanat, Tasarım ve Yaratıcı Kodlama Topluluğunu Bir Araya Getirir
- Yerel Organizatörlerle Dünyanın Birçok Şehrinde Yapılır
- Kapsayıcı ve Herkese Açık: İlk Kez Kod Yazanlar da Dahil
- Bugün: **PCD@Bursa**, Nilüfer İnovasyon Merkezi

São Paulo · New York · Tokyo (PCD @ Worldwide)

// Yaratıcı Kodlama Nedir?

Yaratıcı Kodlama

Kod ile düşünmek, denemek ve üretmek.

Yaratıcı kodlama, kodlama becerilerini görsel, işitsel ve etkileşimli deneyimler üretmek için kullanma pratiğidir.

Burada amaç yalnızca çalışan bir program yazmak değil, kurallar, algoritmalar ve rastlantılar aracılığıyla yeni sonuçlar üretmektir.

Geleneksel Kodlama

"Bir problemi çözmek için kod yazarım."

Yaratıcı Kodlama

"Bir sistem kurarım ve o sistemin üretebileceği sonuçları keşfederim."

Generatif Sanat | Animasyon | Etkileşim | Veri Görselleştirme | Ses | Oyunlar

Örnek Yaratıcı Kodlama Projeleri

GENERATIF SANAT

Substrate

Jared Tarbell · 2003

İncele ↗

Rastgele noktalardan büyüyen çizgiler birbirine çarpıp durur, çarptıkları yerden yeni çizgiler doğar ve kristal ya da şehir planı gibi yapılar oluşur.

Teknoloji Processing

Fikir Basit kural, karmaşık sonuç

ETKILEŞIM

EyeWriter

Zach Lieberman, Theo Watson, Evan Roth ve ekip · 2009

İncele ↗

ALS hastası graffiti sanatçısı Tempt One’ın sadece gözüyle çizim yapmasını sağlayan açık kaynaklı göz takip gözlüğü.

Teknoloji openFrameworks · kızılötesi LED · ucuz kamera

Fikir Kodla erişilebilirlik

SES

Chrome Music Lab

Google Creative Lab · 2016

İncele ↗

Tarayıcıda notalara tıklayarak müziği öğreten, açık kaynaklı deneyler.

Teknoloji Web Audio API · Tone.js

Fikir Algoritma ve oyunla müzik üretimi

OYUN

Townscaper

Oskar Stålberg · 2021

İncele ↗

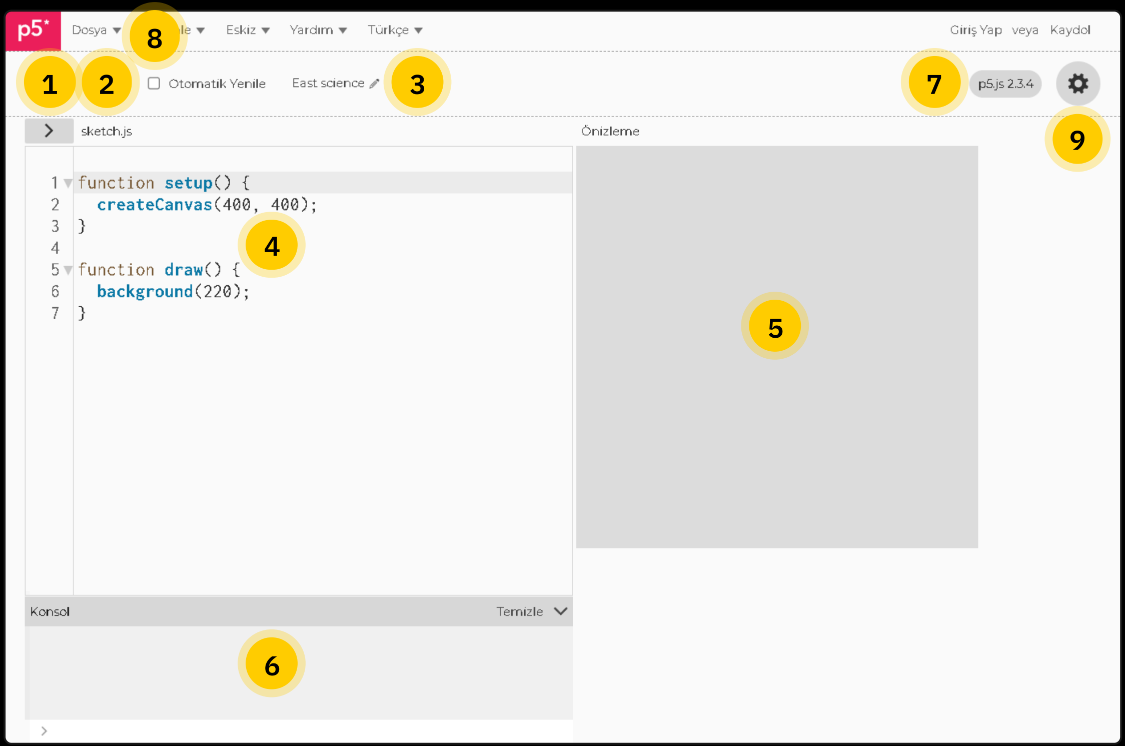
Tıkladıkça uygun bina parçasını algoritmanın seçtiği, rahatlatıcı bir şehir inşa oyuncağı.

Teknoloji Unity · Wave Function Collapse algoritma

Fikir Kural tabanlı prosedürel üretim

// p5.js Temelleri

p5.js Web Arayüzü


p5*
Dosya ▾
le ▾
Eskiz ▾
Yardım ▾
Türkçe ▾
Giriş Yap veya Kaydol
☐ Otomatik Yenile
East science
p5.js 2.3.4
sketch.js
Önizleme
Konsol
Temizle
1
2
3
4
5
6
7
8
9

```
function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
}
```

1. **Play:** Kodu Çalıştır
2. **Stop:** Kodu Durdur
3. **Eskiz Adı**
4. **Kodlama Alanı**
5. **Önizleme**
6. **Konsol**
7. **p5.js Sürümü**
8. **Dosya → Kaydet / Paylaş**
9. **Ayarlar**

editor.p5js.org ↗

Rehberle Birlikte

▣ Türkçe Hızlı Başvuru Rehberi

p5.js Özet Bilgiler (TR) ↗

📄 Basılı rehberinizi yanınızda açık tutun.

Kaynak: Ben Moren · Türkçe Çeviri: Bedirhan Uğur

Yorum Satırları

Program bu satırları çalıştırmaz.
Kodun ne yaptığını açıklamak ve not için kullanışlıdırlar.

TEK SATIR

//

```
// Bu tek satırlık bir yorumdur

line(x1, y1, x2, y2); // bu kısım da çalışmaz
```

İki eğik çizgi yan yana.
Satırın geri kalanı yok sayılır.

ÇOK SATIR

/* */

```
/*
Bu çok satırlık
bir yorumdur
*/

/* beyaz */ rect(20, 20, 60, 40);
```

`/*` ile açılır, `*/` ile kapanır.
Kapanınca aynı satırda kod devam edebilir.

Program Yapısı

Setup bir kez. Draw, o bittikten sonra sürekli çalışır.

setup() / draw()

```
// Eskiz çalışınca bir kez çağrılır
function setup() {
  createCanvas(400, 400); // 160.000 piksellik alan
}

// setup() tamamlandıktan sonra sürekli çalışır
function draw() {
  // bir döngü içinde yürütülen kod
  // varsayılan değer 60 kare/sn
}
```

01 **createCanvas**
Çizim alanı oluşturur

02 **(400, 400)**
Genişlik, yükseklik · piksel

03 **;**
Satırın bittiğini gösterir

04 **p5js.org/reference ↗**
Her fonksiyonun açıklaması

background()

Arka plan rengini ayarlar. draw() her karede tüm tuvali kaplar.

background()

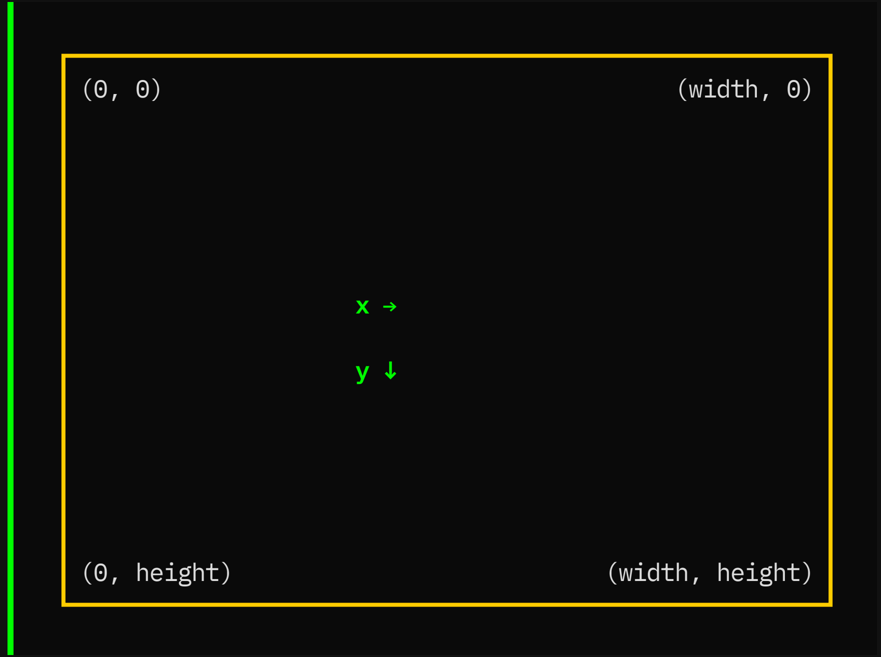
```
function draw() {
  background(20); // gri: 0-255
}
```

01	**background**	Arka plan rengi
02	**0**	Siyah
03	**255**	Beyaz
04	**draw()**	Her karede yeniden boyar

Koordinat Sistemi

Sol üst (0, 0). x sağa gider, y aşağı iner.

width - height


(0, 0)
(width, 0)
x →
y ↓
(0, height)
(width, height)

01 **(0, 0)**
Sol üst köşe

02 **x**
Sağa doğru artar

03 **y**
Aşağı doğru artar

04 **width - height**
Tuvalin genişliği - yüksekliği

2B Şekiller

Çizgi, elips, çember, dikdörtgen, kare, yazı. Sıra: yer, sonra boyut.

line · ellipse · circle · rect · square · text

```
line(20, 80, 200, 80); // çizgi
ellipse(120, 160, 80, 50); // elips
circle(280, 160, 70); // çember
rect(40, 220, 120, 70); // dikdörtgen
square(200, 220, 70); // kare
text("yazı", 40, 60); // yazı
```

01	**line**	x1, y1, x2, y2 | (iki nokta)
02	**ellipse**	x, y, genişlik, yükseklik | (merkez)
03	**circle**	x, y, çap | (merkez)
04	**rect**	x, y, genişlik, yükseklik | (sol üst)
05	**square**	x, y, kenar | (sol üst)
06	**text**	"yazı", x, y | (verilen konuma yazar)

Renk ve Özellikler

Dolgu fill() ile, kontur stroke() ile.

fill

```
fill(120); // gri: 0-255
fill(100, 125, 255); // kırmızı, yeşil, mavi
fill(255, 0, 0, 50); // alfa (saydamlık / opaklık)
fill('red'); // css-svg renk adı
fill('#222222'); // altı haneli hex renk kodu
```

stroke

```
stroke(0, 255, 0); // kontur rengi
strokeWeight(4); // kalınlık, piksel

noFill(); // renk dolgusu yok
noStroke(); // kenar yok
```

fill() kendinden önce çizileni boyamaz.

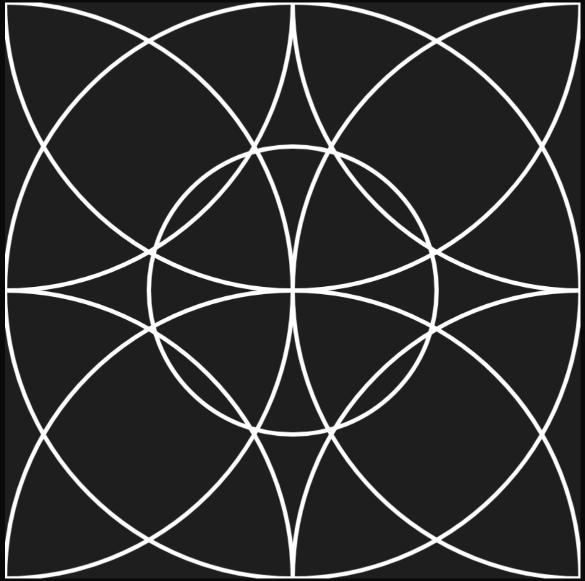
Son çizilen üstte. Katman yok, sıra var.

Öğrendiklerimizi Pekiştirelim

Basit bir uygulama: circle() ile motif oluşturma.

```
function setup() {
  createCanvas(400, 400);
  noFill();
  stroke(255);
  strokeWeight(3);
}

function draw() {
  background(32);

  // köşe daireleri
  circle(0, 0, 400);
  circle(400, 0, 400);
  circle(0, 400, 400);
  circle(400, 400, 400);

  // üst, alt, sol ve sağ daireler
  circle(200, 0, 400);
  circle(200, 400, 400);
  circle(0, 200, 400);
  circle(400, 200, 400);

  // ortadaki daireler
  circle(200, 200, 400);
  circle(200, 200, 200);
}
```

Önizleme **Ortadaki Daireler · 5/5**



Değişken Oluşturma ve Atama

İsim ver, değer sakla — setup() ve draw() dışında tanımla, ikisinde de kullan.

```
let cap = 120; // setup / draw dışı

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(30);
  circle(200, 200, cap);
}
```

01 **let**
Yeni değişken tanımlar

02 **=**
Değer atar (saklar)

03 **global**
setup / draw dışında · her yerde erişilir

04 **setup() içi**
Orada let cap → draw()'da kullanılamaz

let yalnızca bir kez — sonra sadece isim ve **=** ile değer ata.
cap gibi anlamlı isim; sayıyı tekrar tekrar yazma.

Sistem Değişkenleri

Kendi değişkenin değil; p5’nin sürekli güncellediği değerler.

width - height

```
width
height

// tuvalin genişliği - yüksekliği
```

mouseX - mouseY

```
mouseX
mouseY

// anlık yatay - dikey fare konumu
```

Öğrendiklerimizi Pekiştirelim

Mouse ile interaktif çizim örneği.

```
function setup() {
  createCanvas(600, 600);
  background(0);
}

function draw() {
  fill(0);
  stroke(0, 255, 0);
  strokeWeight(2);
  circle(mouseX, mouseY, 50);
}
```

Dene!

```
Yalnızca mouse basılıyken çizmek için:
circle(...) satırını silin veya yorumlayın, sonra:

if (mouseIsPressed) {
  circle(mouseX, mouseY, 50);
}
```

💡 **İpucu** **Çizimi kaydetmek için:**

```
function keyPressed() {
  save("deneme.jpg");
}
```

for Döngüsü

Önce tek tek beş satır, sonra aynı ızgara — bir kural, beş çizgi.

```
// Tek tek
line(0, 0, 0, height);
line(40, 0, 40, height);
line(80, 0, 80, height);
line(120, 0, 120, height);
line(160, 0, 160, height);

// Döngü
for (let i = 0; i < 5; i++) {
  line(i * 40, 0, i * 40, height);
}
```

01 **`i = 0`**
Sayaç sıfırdan başlar

02 **`i < 5`**
Beş kez çalışır

03 **`i++`**
Her turda bir artar

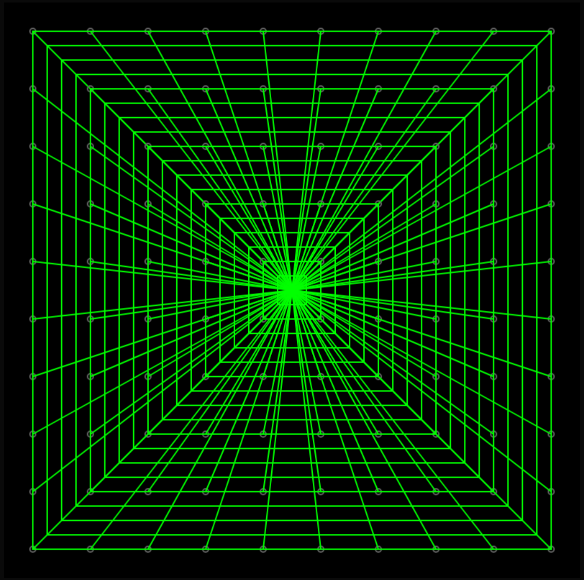
04 **`i * 40`**
Çizgiler 0, 40, 80… diye dizilir

Öğrendiklerimizi Pekiştirelim

Tekrar eden kod yerine döngü; tekrar eden çizimden motif.

```
function setup() {
  createCanvas(400, 400);
  stroke(102);
  noFill();
}

function draw() {
  background(0);
  for (let y = 20; y <= height - 20; y += 40) {
    for (let x = 20; x <= width - 20; x += 40) {
      stroke(102);
      circle(x, y, 4);
      stroke(0, 255, 0);
      line(x, y, width / 2, height / 2);
    }
  }

  rectMode(CENTER);
  stroke(0, 255, 0);
  for (let boyut = height - 40; boyut > 0; boyut -= 20) {
    square(width / 2, height / 2, boyut);
  }
}
```

Önizleme **Merkez Kareleri · 5/5**



random()

Belirtilen aralıkta sayı üretir. Her çalıştırıldığında farklı sonuçlar üretir.

`random(minimum, maksimum)`

```
random(); // 0 ile 1 arası, 1 gelmez
random(0, width); // yatay konum
random(0, 255); // gri

fill(random(0, 255));
ellipse(random(0, width), 200, 40, 40);
```

01 **minimum**
Aralığın altı

02 **maksimum**
Aralığın üstü

03 **her çağrı**
Yeni bir sayı

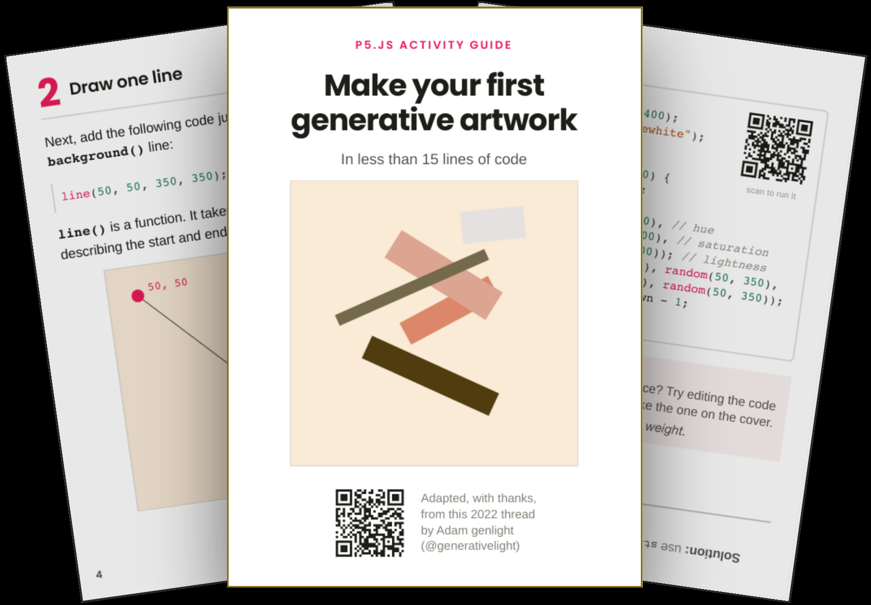
04 **random()**
0 dahil, 1 hariç

Ara ☕

// Uygulamalı Çalışma

Sıra Sizde!

Kendi hızınızda ilerleyin; kitapçık rehberiniz.



P5.JS ACTIVITY GUIDE

Make your first generative artwork

In less than 15 lines of code

Adapted, with thanks, from this 2022 thread by Adam genlight (@generativelight)

2 Draw one line

Next, add the following code ju
`background()` line:

```
line(50, 50, 350, 350);
```

`line()` is a function. It take
describing the start and end

```
400);
ewhite");
0) {
0), // hue
00), // saturation
00)); // lightness
), random(50, 350),
), random(50, 350));
wn - 1;
```

ce? Try editing the code
e the one on the cover.
weight.

Kaynak: PCD Zine Library ↗

Ara ☕

// insan, Makine ve Algoritmalar

Bu Sistemde Kim Ne Yapıyor?

İnsan

Ne yapıyor?

- Fikri ve çerçeveyi belirler
- Kuralları ve sınırları tasarlar
- Hangi verinin kullanılacağına karar verir
- Sonuçları seçer, değiştirir veya reddeder

Yalnızca sonucu değil, sonucun üretileceği sistemi de tasarlar.

Algoritma

Ne yapıyor?

- Belirlenen kurallar içinde olasılıklar üretir
- Tekrar, varyasyon ve hesaplama sağlar
- Gerektiğinde rastlantıyı sürece katar

Tek bir sonucu tarif etmekten çok, olası sonuçların alanını tanımlar.

Makine

Ne yapıyor?

- Dijital bilgiyi fiziksel harekete dönüştürür
- Motorları konum ve hızla yönetir
- Malzeme ve mekanik koşullar sonucu etkiler

Sonuç her zaman tamamen öngörülebilir olmayabilir.

İnsan sistemi tasarlar **Algoritma** olasılıkları üretir **Makine** uygular

Üretim Döngüsü

Süreç doğrusal değil, döngüsel. İnsan → Algoritma → Makine → İnsan

İnsan
Karar · Kural · Seçim

→

Algoritma
Olasılık · Hesaplama · Varyasyon

→

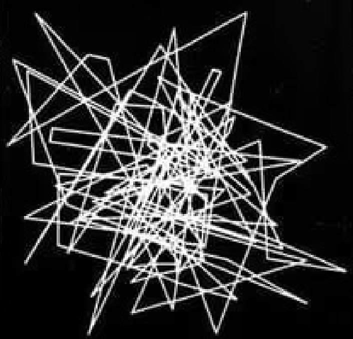
Makine
Fiziksel hareket · Malzeme

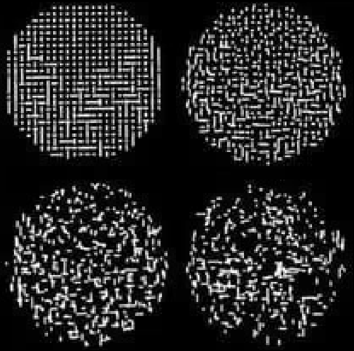
→

Sonuç
Değerlendirilir

↩ Değerlendirme · Seçim · Yeniden düzenleme

Sonuç, insanın değerlendirmesiyle döngüyü yeniden başlatır. Sistem her turda yeniden tanımlanabilir.





Peki bu ilişki bir çizim robotuyla nasıl gösterebiliriz? **Çizim Robotu →**

// Çizim Robotu Gösterimi

// Soru-Cevap ve Açık Tartışma

Öğrenme İpuçları

Her gün biraz kod

15 dakika bile alışkanlık kurar; beceri düzenli tekrarla gelişir.

İncele ve dene

OpenProcessing’de beğendiğin eskizleri aç; kodu okuyup küçük değişikliklerle kendi denemeni yap.

Hatadan korkma

Konsoldaki her mesaj, konuyu derinleştirmek için bir ipucudur.

Adım adım geliştir

Önce basit bir eskizle başla; zamanla hareket, renk ve etkileşim ekleyerek büyüt.

Paylaş

Çalışmalarını toplulukla paylaş; geri bildirim öğrenmeyi hızlandırır.

Kaynaklar

ARAÇLAR · ÖĞRENME

p5.js Web Editor
Tarayıcı Editörü

Processing
Masaüstü IDE

p5.js Reference
Fonksiyon Sözlüğü

p5.js Learn
Adım Adım Dersler

p5.js Cheat Sheet
EN ↗ · TR ↗ · Hızlı Başvuru

TOPLULUK · PAYLAŞIM

OpenProcessing
Eskiz Galerisi

PCD Zine Library
Atölye Kitapçıkları

Processing Foundation
Topluluk Vakfı

Processing Community Day
PCD Etkinlik Ağı

Processing Forum
Topluluk desteği

💾 Bugünkü Atölye Sunumu

▣ **PCD @BURSA · 2026**

bedirxanugur.github.io/yaratici-kodlama ↗

Telefonunuzla QR’ı tarayın veya bağlantıyı açın.

Katılımınız için Teşekkürler!

PCD @Bursa 2026 Yaratıcı Kodlama Atölyesi

NİLÜFER
BELEDİYESİ
BURSA

İNOVASYON
MERKEZİ